

重慶理工大學

深度学习

学　　院　计算机科学与工程学院

学　　号　_____

姓　　名　_____

指 导 教 师　_____

时　　间　_____

实验 4 神经网络模型应用

实验目的：

1）掌握神经网络模型的实现过程。
2）熟悉 PyTorch 编程。
3）学习使用 PyTorch 编程实现神经网络。

实验内容：

Ex1 补全一层全连接神经网络示例，使用 PyTorch 自动求导和反向传播完成一次参数更新，并说明程序完成的工作。

Ex2 使用 PyTorch 搭建神经网络，对 MNIST 手写数字数据集进行分类，并在测试集上计算分类准确率。

Ex1 实现思路：

输入张量 x 的形状为 10×3，目标张量 y 的形状为 10×2。全连接层 `nn.Linear(3, 2)` 为每个样本计算 $xW^{\mathsf{T}} + b$，输出形状为 10×2。程序使用均方误差衡量预测与目标之间的差异，调用 `backward()` 根据计算图求出权重和偏置的梯度，再由学习率为 0.01 的随机梯度下降优化器更新参数。最后重新前向计算，比较更新前后的损失。

整个程序实现了一层全连接网络从前向传播到参数更新的基本过程。为了便于重复实验，程序固定了随机种子，使每次运行时的输入数据和初始参数保持一致。

Ex1 源代码：

```
"""    4-1    PyTorch                    """

import torch
from torch import nn, optim


def main():
    #
    torch.manual_seed(7)

    # 10                3           2
    x = torch.randn(10, 3)
    y = torch.randn(10, 2)

    #          3           2       y_pred = xW^T + b
    linear = nn.Linear(3, 2)
    print("    w  ", linear.weight)
    print("    b  ", linear.bias)

    #                           SGD        0.01
```

```
    criterion = nn.MSELoss()
    optimizer = optim.SGD(linear.parameters(), lr=0.01)

    #                              10
    pred = linear(x)

    #                  autograd
    loss = criterion(pred, y)
    print("          ", loss.item())
    optimizer.zero_grad()
    loss.backward()

    #
    print("dL/dw  ", linear.weight.grad)
    print("dL/db  ", linear.bias.grad)

    #                  no_grad
    optimizer.step()
    with torch.no_grad():
        updated_pred = linear(x)
        updated_loss = criterion(updated_pred, y)

    print("          ", updated_loss.item())


if __name__ == "__main__":
    main()
```

运行（测试）过程及结果：

```
PyTorch 1.13.1          7
          1.3483909368515015
dL/dw  tensor([[-0.3902, -0.2571, -0.1136],
               [-0.5928,  0.7947,  0.6030]])
dL/db  tensor([0.2091, 1.0205])
          1.322024941444397
```

从运行结果可以看到，经过一次参数更新，均方误差由 1.3484 下降至 1.3220。虽然损失下降幅度不大，但说明反向传播计算出的梯度已经用于参数更新，使模型的预测结果更接近目标值。

Ex2 实现思路：

MNIST 数据集中的图像均为 28×28 像素的灰度图。为了便于神经网络处理，程序首先将图像转换为张量并按数据集的均值和标准差进行标准化，然后将其展平为 784 维向量，作为全连接网络的输入。分类器由 784 到 128 的全连接层、ReLU 激活函数和 128 到 10 的输出层组成，输出的 10 个分数对应数字 0 至 9。训练时使用交叉熵损失和带动量的随机梯

度下降，训练 3 轮；最后在独立测试集上统计分类准确率。

Ex2 源代码：

```
"""    4-2    PyTorch              MNIST            """

import torch
from torch import nn, optim
from torch.utils.data import DataLoader
from torchvision import datasets, transforms


def main():
    #
    torch.manual_seed(7)

    #     CPU       PyTorch 1.13.1    macOS  Windows    Linux
    device = torch.device("cpu")

    # MNIST    28×28               [0, 1]
    transform = transforms.Compose(
        [
            transforms.ToTensor(),
            transforms.Normalize((0.1307,), (0.3081,)),
        ]
    )
    training_dataset = datasets.MNIST(
        root="data", train=True, download=True, transform=transform
    )
    test_dataset = datasets.MNIST(
        root="data", train=False, download=True, transform=transform
    )
    training_loader = DataLoader(
        training_dataset, batch_size=64, shuffle=True, num_workers=0
    )
    test_loader = DataLoader(
        test_dataset, batch_size=1000, shuffle=False, num_workers=0
    )

    #    28×28          784                    10
    model = nn.Sequential(
        nn.Flatten(),
        nn.Linear(28 * 28, 128),
        nn.ReLU(),
        nn.Linear(128, 10),
    ).to(device)
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.SGD(model.parameters(), lr=0.01, momentum=0.9)
    number_of_epochs = 3
```

```
    print("    4-2  MNIST          ")
    print("PyTorch      ", torch.__version__)
    print("        ", device)
    print(
        "          {}           {}          64          {}".format(
            len(training_dataset), len(test_dataset), number_of_epochs
        )
    )

    for epoch_number in range(1, number_of_epochs + 1):
        model.train()
        total_loss = 0.0
        for images, labels in training_loader:
            images = images.to(device)
            labels = labels.to(device)

            optimizer.zero_grad()
            class_scores = model(images)
            loss = criterion(class_scores, labels)
            loss.backward()
            optimizer.step()
            total_loss += loss.item() * labels.size(0)

        average_loss = total_loss / len(training_dataset)
        print(
            "   {}           {:.4f}".format(epoch_number, average_loss)
        )

    #
    model.eval()
    correct_count = 0
    with torch.no_grad():
        for images, labels in test_loader:
            images = images.to(device)
            labels = labels.to(device)
            predicted_labels = model(images).argmax(dim=1)
            correct_count += (predicted_labels == labels).sum().item()

    accuracy = 100.0 * correct_count / len(test_dataset)
    print(
        "            {} / {}        {:.2f}%".format(
            correct_count, len(test_dataset), accuracy
        )
    )


if __name__ == "__main__":
    main()
```

运行（测试）过程及结果：

```
    4-2  MNIST
PyTorch       1.13.1
         cpu
          60000            10000          64          3
  1           0.2755
  2           0.1165
  3           0.0796
            9724 / 10000         97.24%
```

从三轮训练结果看，训练损失由第一轮的 0.2755 降至第三轮的 0.0796，说明模型对训练数据的拟合逐渐改善。测试集中，模型正确分类了 10,000 张图像中的 9,724 张，准确率为 97.24%，整体上能够较好地识别手写数字。

总结：

本次实验分别完成了一层全连接网络的参数更新和 MNIST 手写数字分类任务。通过第一个实验，了解了 PyTorch 自动求导和反向传播的基本过程，并通过比较参数更新前后的损失，观察到梯度下降对模型参数的调整作用。

在第二个实验中，使用 PyTorch 搭建了一个包含隐藏层的全连接神经网络，对 MNIST 数据集进行训练和测试。随着训练轮数增加，模型的训练损失逐渐下降，最终在测试集上达到了 97.24% 的分类准确率，说明该网络能够较好地完成手写数字识别任务。

通过这两个实验，进一步熟悉了使用 PyTorch 构建模型、计算损失、更新参数以及评估模型性能的基本流程。此次实验采用的网络结构比较简单，后续还可以尝试调整隐藏层规模或训练轮数，观察不同设置对分类结果的影响。